

BAB II

FONDASI PROYEK

2.1 Instalasi Proyek Laravel dan Konfigurasi Basis Data

2.1.1) Perangkat yang Digunakan

- XAMPP atau Laragon yang sudah berjalan (Apache/Nginx + MariaDB/MySQL)
- Composer versi terbaru
- PHP 8.2 atau lebih baru
- Editor kode (VS Code)
- Koneksi internet

2.1.2) Periksa versi PHP

Buka terminal, lalu jalankan :

```
php -v  
composer -V
```

Pastikan PHP menunjukkan versi 8.2 atau lebih tinggi. Bila masih 8.1 atau di bawahnya, Laravel 12 keatas tidak akan bisa dipasang. Laporkan ke guru sebelum melanjutkan.

2.1.3) Buat proyek baru

Masuk ke folder tempat proyek disimpan, lalu:

```
composer create-project laravel/laravel peminjaman-alat  
cd peminjaman-alat
```

2.1.4) Pasang paket yang dibutuhkan

Pada terminal, unduh paket yang dibutuhkan :

```
composer require laravel/fortify spatie/laravel-permission barryvdh/laravel-dompdf  
composer require --dev barryvdh/laravel-debugbar
```

Daftar paket dan alasan pemakainya :

Paket	Lingkungan	Alasan
laravel/fortify	production	Menyediakan mesin autentikasi (login, logout, pembatasan percobaan) tanpa memaksakan tampilan bawaan
spatie/laravel-permission	production	Mengelola peran dan izin tiga level
barryvdh/laravel-dompdf	production	Membuat laporan PDF tanpa perlu Node.js
barryvdh/laravel-debugbar	dev	Menampilkan jumlah query dan waktu muat halaman

2.1.5) Terbitkan berkas konfigurasi paket

Pada terminal, jalankan perintah berikut :

```
php artisan vendor:publish --provider="Laravel\Fortify\FortifyServiceProvider"
php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvider"
```

Perintah pertama menghasilkan `config/fortify.php`, folder `app/Actions/Fortify/`, dan berkas `app/Providers/FortifyServiceProvider.php`. Perintah kedua menghasilkan `config/permission.php` dan satu migration untuk lima tabel peran dan izin.

2.1.6) Daftarkan FortifyServiceProvider

Buka `bootstrap/providers.php` dan tambahkan barisnya:

```
<?php

return [
    App\Providers\AppServiceProvider::class,
    App\Providers\FortifyServiceProvider::class,
];
```

Tanpa baris ini, seluruh konfigurasi yang Anda tulis tidak akan pernah dijalankan.

2.1.7) Hapus migration dua faktor

Fortify menerbitkan satu migration yang menambahkan kolom autentikasi dua faktor ke tabel `users`. Fitur itu tidak dipakai, dan bila dibiarkan, tabel `users` tidak akan lagi cocok dengan kamus data yang Anda susun.

Buka folder `database/migrations/`, cari berkas berakhiran `_add_two_factor_columns_to_users_table.php`, lalu **hapus berkas tersebut**.

2.1.8) Sesuaikan konfigurasi MariaDB

Buka `config/database.php`, cari bagian `mysql`, dan pastikan dua baris berikut bernilai persis seperti ini:

```
'charset' => 'utf8mb4',
'collation' => 'utf8mb4_unicode_ci',
```

Ini wajib. Laravel versi baru kadang memakai nilai bawaan `utf8mb4_0900_ai_ci`. Nilai itu hanya dikenal MySQL 8 dan akan membuat MariaDB menolak seluruh migration dengan pesan `unknown collation`.

Selanjutnya buka berkas `.env` dan sesuaikan:

```
APP_NAME="Peminjaman Alat"
APP_TIMEZONE=Asia/Jakarta
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=peminjaman_alat
DB_USERNAME=root
DB_PASSWORD=
```

2.1.9) Buat basis data kosong

Buka phpMyAdmin di <http://localhost/phpmyadmin>, lalu buat basis data baru:

- Nama: `peminjaman_alat`
- Collation: `utf8mb4_unicode_ci`

Jangan membuat satu tabel pun secara manual. Seluruh tabel akan dibuat lewat migration.

2.1.10) Uji koneksi

Pada terminal, jalankan perintah ini:

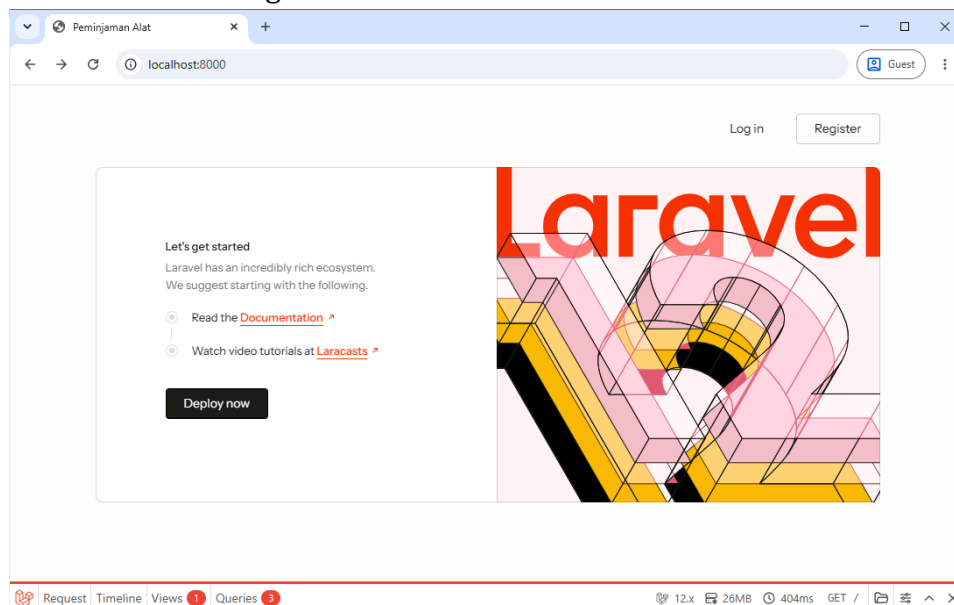
```
php artisan migrate:fresh
```

Hasil yang diharapkan: migrasi bawaan Laravel dan migrasi tabel peran dari paket otorisasi berjalan tanpa kesalahan. Periksa di phpMyAdmin, basis data kini berisi tabel bawaan (`users`, `sessions`, `cache`, `jobs`, dan seterusnya) serta lima tabel peran (`roles`, `permissions`, `model_has_roles`, `model_has_permissions`, `role_has_permissions`).

Terakhir, jalankan server pengembangan di terminal:

```
php artisan serve
```

Buka <http://127.0.0.1:8000>. Halaman sambutan Laravel harus tampil, dan di bagian bawah layar muncul bilah Debugbar.



2.2 Migration Delapan Tabel dan Model Eloquent

2.2.1) Sesuaikan tabel users

Tabel users sudah ada sebagai bawaan Laravel, tetapi kolomnya belum sesuai kamus data. Buka berkas `database/migrations/0001_01_01_000000_create_users_table.php` dan ubah hanya bagian `Schema::create('users', ...)`. Dua tabel lain di berkas itu (`password_reset_tokens` dan `sessions`) biarkan apa adanya.

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('nama', 100);
    $table->string('username', 50)->unique();
    $table->string('email', 100)->nullable()->unique();
    $table->string('password');
    $table->string('no_telp', 20)->nullable();
    $table->boolean('is_aktif')->default(true);
    $table->rememberToken();
    $table->timestamps();
});
```

Kolom email boleh kosong dan tetap berindeks unik. MariaDB mengizinkan banyak baris bernilai NULL pada indeks unik, jadi peminjam yang tidak punya email tidak akan bertabrakan satu sama lain.

2.2.2) Buat tujuh berkas migration sekaligus

Pada terminal jalankan berurutan, jangan diacak, karena urutan inilah yang menentukan urutan pembuatan tabel:

```
php artisan make:migration create_kategori_table
php artisan make:migration create_alat_table
php artisan make:migration create_peminjaman_table
php artisan make:migration create_detail_peminjaman_table
php artisan make:migration create_pengembalian_table
php artisan make:migration create_log_aktivitas_table
php artisan make:migration create_pengaturan_table
```

Nama berkas migration tetap memakai pola bahasa Inggris `create_..._table` karena Artisan memakai pola itu untuk menebak nama tabel secara otomatis. Selebihnya — nama tabel, nama kolom, nama variabel, dan nama method — seluruhnya berbahasa Indonesia.

2.2.3) Isi migration kategori

Buka file `..._create_kategori_table.php`:

```
public function up(): void
{
    Schema::create('kategori', function (Blueprint $table) {
        $table->id();
        $table->string('nama', 100)->unique();
        $table->text('deskripsi')->nullable();
        $table->timestamps();
    });
}
```

```

public function down(): void
{
    Schema::dropIfExists('kategori');
}

```

Kolom `created_at` dan `updated_at` tidak digambar di ERD karena bukan bagian dari model bisnis, tetapi tetap dibuat agar fitur bawaan Eloquent berjalan normal.

2.2.4) Isi migration alat

Berkas ini adalah yang pertama memuat foreign key dan CHECK constraint. Jangan lupa menambahkan use `Illuminate\Support\Facades\DB`; di bagian atas berkas.

Buka file `..._create_alat_table.php` :

```

3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\DB;
6  use Illuminate\Support\Facades\Schema;
7
8  return new class extends Migration
9  {
10     public function up(): void
11     {
12         Schema::create('alat', function (Blueprint $table) {
13             $table->id();
14             $table->foreignId('kategori_id')
15                 ->constrained('kategori')
16                 ->restrictOnDelete();
17             $table->string('kode_alat', 30)->unique();
18             $table->string('nama', 150);
19             $table->text('deskripsi')->nullable();
20             $table->integer('stok')->default(0);
21             $table->integer('stok_tersedia')->default(0);
22             $table->enum('kondisi', ['baik', 'rusak_ringan', 'rusak_berat'])
23                 ->default('baik');
24             $table->string('foto')->nullable();
25             $table->timestamps();
26         });
27
28         DB::statement('ALTER TABLE alat
29             ADD CONSTRAINT chk_alat_stok CHECK (stok ≥ 0)');
30
31         DB::statement('ALTER TABLE alat
32             ADD CONSTRAINT chk_alat_tersedia
33             CHECK (stok_tersedia ≥ 0 AND stok_tersedia ≤ stok)');
34     }
35
36     public function down(): void
37     {
38         Schema::dropIfExists('alat');
39     }
40 };

```

Tiga hal yang perlu dipahami dari kode di atas:

Bagian	Alasan
<code>constrained('kategori')</code>	Nama tabel ditulis eksplisit. Tanpa itu Laravel menebak tabelnya bernama kategoris, dan migration gagal
<code>restrictOnDelete()</code>	Kategori yang masih dipakai alat tidak dapat dihapus.
<code>DB::statement</code> untuk CHECK	Schema Builder Laravel tidak menyediakan method untuk CHECK constraint, jadi ditulis sebagai SQL mentah

CHECK aturan $0 \leq \text{stok_tersedia} \leq \text{stok_dijaga}$ langsung oleh basis data, sehingga tetap berlaku walaupun ada kesalahan di kode PHP.

2.2.5) Isi migration peminjaman

Buka file `..._create_peminjaman_table.php`:

```
public function up(): void
{
    Schema::create('peminjaman', function (Blueprint $table) {
        $table->id();
        $table->string('kode_pinjam', 20)->unique();
        $table->foreignId('user_id')
            ->constrained('users')
            ->restrictOnDelete();
        $table->foreignId('petugas_id')
            ->nullable()
            ->constrained('users')
            ->nullOnDelete();
        $table->date('tgl_pinjam');
        $table->date('tgl_harus_kembali');
        $table->date('tgl_diajukan_kembali')->nullable();
        $table->enum('status', [
            'diajukan',
            'ditolak',
            'dipinjam',
            'menunggu_verifikasi',
            'selesai',
        ])->default('diajukan')->index();
        $table->text('keperluan')->nullable();
        $table->text('alasan_tolak')->nullable();
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('peminjaman');
}
```

Tabel ini punya **dua** foreign key ke tabel yang sama. `user_id` adalah peminjamnya, `petugas_id` adalah petugas yang memproses. `petugas_id` boleh kosong karena pengajuan yang baru masuk belum diproses siapa pun.

Kolom `status` diberi `->index()` karena hampir setiap halaman daftar menyaring berdasarkan kolom ini.

2.2.6) Isi migration detail_peminjaman

Buka file `..._create_detail_peminjaman_table.php`, dan tambahkan use `Illuminate\Support\Facades\DB`:

```
public function up(): void
{
    Schema::create('detail_peminjaman', function (Blueprint $table) {
        $table->id();
        $table->foreignId('peminjaman_id')
            ->constrained('peminjaman')
            ->cascadeOnDelete();
        $table->foreignId('alat_id')
            ->constrained('alat')
            ->restrictOnDelete();
        $table->integer('jumlah');
        $table->enum('kondisi_kembali', [
            'baik',
            'rusak_ringan',
            'rusak_berat',
            'hilang',
        ]->nullable());
        $table->decimal('denda', 12, 2)->default(0);
        $table->timestamps();

        $table->unique(['peminjaman_id', 'alat_id']);
    });

    DB::statement('ALTER TABLE detail_peminjaman
    ADD CONSTRAINT chk_detail_jumlah CHECK (jumlah > 0)');
}

public function down(): void
{
    Schema::dropIfExists('detail_peminjaman');
}
```

Dua pilihan penting di sini:

- `cascadeOnDelete()` ke `peminjaman` — bila data induk dihapus, baris detailnya ikut terhapus. Ini aman karena hanya mengizinkan penghapusan pengajuan yang belum pernah disetujui.
- `restrictOnDelete()` ke `alat` — alat yang pernah dipinjam tidak boleh dihapus, karena riwayat transaksi harus tetap utuh.
- `unique(['peminjaman_id', 'alat_id'])` — Satu alat hanya boleh muncul satu kali dalam satu pengajuan.

Nilai `kondisi_kembali` sengaja kosong sampai alat benar-benar dikembalikan. Nilai kosong inilah yang dibaca untuk memutuskan alat mana yang kembali ke stok.

2.2.7) Isi migration pengembalian

Buka file `..._create_pengembalian_table.php` :

```
public function up(): void
{
    Schema::create('pengembalian', function (Blueprint $table) {
        $table->id();
        $table->foreignId('peminjaman_id')
            ->unique()
            ->constrained('peminjaman')
            ->restrictOnDelete();
        $table->foreignId('petugas_id')
            ->constrained('users')
            ->restrictOnDelete();
        $table->date('tgl_kembali');
        $table->integer('hari_terlambat')->default(0);
        $table->decimal('denda', 12, 2)->default(0);
        $table->decimal('denda_kerusakan', 12, 2)->default(0);
        $table->decimal('total_denda', 12, 2)->default(0);
        $table->text('catatan')->nullable();
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('pengembalian');
}
```

`peminjaman_id` diberi `->unique()` karena relasinya satu ke satu: satu pengajuan hanya boleh menghasilkan satu baris pengembalian. Tanpa indeks unik ini, petugas yang tidak sengaja menekan tombol verifikasi dua kali akan membuat denda terhitung dua kali.

Tiga kolom denda semuanya bernilai bawaan nol, karena isinya akan ditulis oleh trigger, bukan diketik manusia.

2.2.8) Isi migration log aktivitas dan pengaturan

Buka file `..._create_log_aktivitas_table.php` :

```
public function up(): void
{
    Schema::create('log_aktivitas', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')
            ->nullable()
            ->constrained('users')
            ->nullOnDelete();
        $table->string('aksi', 50);
        $table->string('tabel_tujuan', 50)->nullable();
        $table->text('deskripsi')->nullable();
        $table->string('ip_address', 45)->nullable();
        $table->timestamp('created_at')->useCurrent()->index();
    });
}

public function down(): void
{
    Schema::dropIfExists('log_aktivitas');
}
```

Tabel ini hanya punya `created_at`, tanpa `updated_at`, karena catatan log tidak pernah diubah. Kolom `user_id` boleh kosong supaya percobaan login yang gagal — yang pelakunya belum diketahui — tetap dapat dicatat.

Buka file `..._create_pengaturan_table.php`:

```
public function up(): void
{
    Schema::create('pengaturan', function (Blueprint $table) {
        $table->id();
        $table->string('kunci', 50)->unique();
        $table->string('nilai', 255);
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('pengaturan');
}
```

2.2.9) Jalankan migration

Pada terminal jalan migration :

```
php artisan migrate:fresh
```

Perintah `migrate:fresh` menghapus seluruh tabel lalu membuatnya ulang dari awal. Aman dipakai sekarang karena belum ada satu pun data penting.

Buka phpMyAdmin dan periksa: harus ada 8 tabel domain, 5 tabel peran dan izin, serta tabel infrastruktur bawaan Laravel.

Uji juga bahwa CHECK constraint benar-benar bekerja. Jalankan perintah berikut di tab SQL phpMyAdmin:

```
INSERT INTO kategori (nama, created_at, updated_at)
VALUES ('Uji Coba', NOW(), NOW());
```

```
INSERT INTO alat (kategori_id, kode_alat, nama, stok, stok_tersedia, kondisi, created_at,
updated_at)
VALUES (1, 'UJI-001', 'Alat Uji', 5, 9, 'baik', NOW(), NOW());
```

Perintah kedua **harus gagal**, karena `stok_tersedia` bernilai 9 sementara stok hanya 5. Bila justru berhasil, CHECK constraint Anda tidak terpasang; periksa kembali Langkah 4. Setelah terbukti, jalankan lagi `php artisan migrate:fresh` untuk membersihkan data uji.

2.2.10) Buat model Eloquent

Pada terminal buat model :

```
php artisan make:model Kategori
php artisan make:model Alat
php artisan make:model Peminjaman
php artisan make:model DetailPeminjaman
php artisan make:model Pengembalian
php artisan make:model LogAktivitas
php artisan make:model Pengaturan
```

Laravel menebak nama tabel dengan menambahkan huruf **s** pada nama model. Karena seluruh tabel kita berbahasa Indonesia dan tidak berbentuk jamak, **setiap model wajib menyebut nama tabelnya secara eksplisit.**

Buka file `app\Models\Kategori.php` :

```
7 class Kategori extends Model
8 {
9     protected $table = 'kategori';
10
11     protected $fillable = ['nama', 'deskripsi'];
12 }
13
```

Buka file `app\Models\Alat.php` :

```
7 class Alat extends Model
8 {
9     protected $table = 'alat';
10
11     protected $fillable = [
12         'kategori_id', 'kode_alat', 'nama', 'deskripsi',
13         'stok', 'stok_tersedia', 'kondisi', 'foto',
14     ];
15 }
16
```

Buka file `app\Models\Peminjaman.php` :

```
5 use Illuminate\Database\Eloquent\Model;
6
7 class Peminjaman extends Model
8 {
9     protected $table = 'peminjaman';
10
11     protected $fillable = [
12         'kode_pinjam', 'user_id', 'petugas_id', 'tgl_pinjam',
13         'tgl_harus_kembali', 'tgl_diajukan_kembali', 'status',
14         'keperluan', 'alasan_tolak',
15     ];
16
17     protected $casts = [
18         'tgl_pinjam'           => 'date',
19         'tgl_harus_kembali'    => 'date',
20         'tgl_diajukan_kembali' => 'date',
21     ];
22 }
```

Buka file `app\Models\DetailPeminjaman.php`:

```
7 class DetailPeminjaman extends Model
8 {
9     protected $table = 'detail_peminjaman';
10
11     protected $fillable = [
12         'peminjaman_id', 'alat_id', 'jumlah', 'kondisi_kembali', 'denda',
13     ];
14 }
15
```

Buka file `app\Models\Pengembalian.php`:

```
7 class Pengembalian extends Model
8 {
9     protected $table = 'pengembalian';
10
11     protected $fillable = [
12         'peminjaman_id', 'petugas_id', 'tgl_kembali',
13         'denda_kerusakan', 'catatan',
14     ];
15
16     protected $casts = [
17         'tgl_kembali' => 'date',
18     ];
19 }
```

Buka file `app\Models\LogAktivitas.php`:

```
7 class LogAktivitas extends Model
8 {
9     protected $table = 'log_aktivitas';
10
11     public $timestamps = false;
12
13     protected $fillable = [
14         'user_id', 'aksi', 'tabel_tujuan', 'deskripsi', 'ip_address',
15     ];
16 }
```

Buka file `app\Models\Pengaturan.php`:

```
7 class Pengaturan extends Model
8 {
9     protected $table = 'pengaturan';
10
11     protected $fillable = ['kunci', 'nilai'];
12
13     public static function ambil(string $kunci, $bawaan = null)
14     {
15         $baris = static::where('kunci', $kunci)→first();
16
17         return $baris ? $baris→nilai : $bawaan;
18     }
19 }
```

`public $timestamps = false` pada `LogAktivitas` wajib ada. Tanpa itu Eloquent akan mencari kolom `updated_at` yang memang sengaja tidak dibuat, dan setiap penulisan log akan gagal.

Method `ambil()` pada `Pengaturan` akan dipakai berkali-kali. Nilainya sengaja dibaca langsung dari basis data tanpa cache, supaya selalu sama dengan nilai yang dibaca trigger.

Relasi antar model belum ditulis sekarang. Masing-masing akan ditambahkan pada job sheet yang membutuhkannya.

2.3 Peran, Izin, dan Seeder Data Awal

2.3.1) Siapkan model User

Buka file `app\Models\User.php` dan tambahkan trait dari paket otorisasi, lalu sesuaikan kolom yang boleh diisi:

```
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Foundation\Auth\User as Authenticatable;
7 use Illuminate\Notifications\Notifiable;
8 use Spatie\Permission\Traits\HasRoles;
9
10 class User extends Authenticatable
11 {
12     use HasFactory, Notifiable, HasRoles;
13
14     protected $fillable = [
15         'nama', 'username', 'email', 'password', 'no_telp', 'is_aktif',
16     ];
17
18     protected $hidden = [
19         'password',
20         'remember_token',
21     ];
22
23     protected function casts(): array
24     {
25         return [
26             'is_aktif' => 'boolean',
27             'password' => 'hashed',
28         ];
29     }
30 }
```

Trait `HasRoles` memberi model ini method `assignRole()`, `hasRole()`, dan `can()` yang akan dipakai di hampir seluruh job sheet berikutnya.

Cast `'password' => 'hashed'` membuat Laravel otomatis mengubah password menjadi hash bcrypt setiap kali kolom itu diisi. Karena itu, mulai sekarang **jangan pernah menulis `Hash::make()` secara manual** — password akan ter-hash dua kali dan login selalu gagal.

2.3.2) Daftarkan alias middleware

Buka `bootstrap/app.php` dan lengkapi bagian `withMiddleware`:

```
3 use Illuminate\Foundation\Application;
4 use Illuminate\Foundation\Configuration\Exceptions;
5 use Illuminate\Foundation\Configuration\Middleware;
6 use Spatie\Permission\Middleware\PermissionMiddleware;
7 use Spatie\Permission\Middleware\RoleMiddleware;
8 use Spatie\Permission\Middleware\RoleOrPermissionMiddleware;
9
10 return Application::configure(basePath: dirname(__DIR__))
11     →withRouting(
12         web: __DIR__.'/../routes/web.php',
13         commands: __DIR__.'/../routes/console.php',
14         health: 'up',
15     )
16     →withMiddleware(function (Middleware $middleware): void {
17         $middleware→alias([
18             'role' => RoleMiddleware::class,
19             'permission' => PermissionMiddleware::class,
20             'role_or_permission' => RoleOrPermissionMiddleware::class,
21         ]);
22     })
23     →withExceptions(function (Exceptions $exceptions): void {
24         //
25     })→create();
```

Tanpa pendaftaran ini, penulisan `middleware('permission:alat.kelola')` pada route akan menghasilkan kesalahan *target class does not exist*.

2.3.3) Buat seeder peran dan izin

Buka terminal buat seeder :

```
php artisan make:seeder PeranIzinSeeder
```

Isi file `database/seeder/PeranIzinSeeder.php`:

```
5 use Illuminate\Database\Seeder;
6 use Spatie\Permission\Models\Permission;
7 use Spatie\Permission\Models\Role;
8
9 class PeranIzinSeeder extends Seeder
10 {
11     public function run(): void
12     {
13         $daftarIzin = [
14             // Izin milik Admin
15             'user.kelola',
16             'alat.kelola',
17             'kategori.kelola',
18             'peminjaman.kelola',
19             'pengembalian.kelola',
20             'log.lihat',
21             'pengaturan.kelola',
22
23             // Izin milik Petugas
24             'peminjaman.setujui',
25             'pengembalian.pantau',
26             'laporan.cetak',
27
28             // Izin milik Peminjam
29             'alat.lihat',
30             'peminjaman.ajukan',
31             'peminjaman.kembalikan',
32         ];
33
34         foreach ($daftarIzin as $namaIzin) {
35             Permission::firstOrCreate([
36                 'name' => $namaIzin,
37                 'guard_name' => 'web',
38             ]);
39         }
40
41         $peranAdmin = Role::firstOrCreate(['name' => 'admin', 'guard_name' => 'web']);
42         $peranPetugas = Role::firstOrCreate(['name' => 'petugas', 'guard_name' => 'web']);
43         $peranPeminjam = Role::firstOrCreate(['name' => 'peminjam', 'guard_name' => 'web']);
44
45         $peranAdmin->syncPermissions([
46             'user.kelola',
47             'alat.kelola',
48             'kategori.kelola',
49             'peminjaman.kelola',
50             'pengembalian.kelola',
51             'log.lihat',
52             'pengaturan.kelola',
53         ]);
54
55         $peranPetugas->syncPermissions([
56             'peminjaman.setujui',
57             'pengembalian.pantau',
58             'laporan.cetak',
59         ]);
60     }
61 }
```

```

61     $peranPeminjam→syncPermissions([
62         'alat.lihat',
63         'peminjaman.ajukan',
64         'peminjaman.kembalikan',
65     ]);
66 }
67 }

```

Perhatikan pemetaannya terhadap matriks hak akses. Tidak ada satu pun izin yang dimiliki dua peran sekaligus. Admin **tidak** memiliki `peminjaman.setujui`, dan Petugas **tidak** memiliki `alat.kelola`.

`firstOrCreate` dan `syncPermissions` dipakai supaya seeder aman dijalankan berulang kali. `syncPermissions` mengganti seluruh daftar izin peran dengan daftar baru, jadi bila nanti ada izin yang dihapus dari daftar, izin itu ikut terlepas dari perannya.

2.3.4) Buat seeder pengguna

Buka terminal buat seeder :

```
php artisan make:seeder PenggunaSeeder
```

Buka file `database\seeders\PenggunaSeeder.php` :

```

5  use App\Models\User;
6  use Illuminate\Database\Seeder;
7
8  class PenggunaSeeder extends Seeder
9  {
10     public function run(): void
11     {
12         $daftarPengguna = [
13             [
14                 'nama'      => 'Administrator',
15                 'username' => 'admin',
16                 'email'    => 'admin@sekolah.sch.id',
17                 'no_telp'  => '081200000001',
18                 'peran'    => 'admin',
19             ],
20             [
21                 'nama'      => 'Petugas Laboratorium',
22                 'username' => 'petugas',
23                 'email'    => 'petugas@sekolah.sch.id',
24                 'no_telp'  => '081200000002',
25                 'peran'    => 'petugas',
26             ],
27             [
28                 'nama'      => 'Siswa Peminjam',
29                 'username' => 'peminjam',
30                 'email'    => 'peminjam@sekolah.sch.id',
31                 'no_telp'  => '081200000003',
32                 'peran'    => 'peminjam',
33             ],
34         ];

```

```

36         foreach ($daftarPengguna as $data) {
37             $pengguna = User::firstOrCreate(
38                 ['username' => $data['username']],
39                 [
40                     'nama' => $data['nama'],
41                     'email' => $data['email'],
42                     'no_telp' => $data['no_telp'],
43                     'password' => 'password123',
44                     'is_aktif' => true,
45                 ]
46             );
47             $pengguna->syncRoles([$data['peran']]);
48         }
49     }
50 }
51 }

```

Password ditulis polos di seeder, dan itu tidak masalah: cast `'password' => 'hashed'` pada model User mengubahnya menjadi hash bcrypt sebelum disimpan. Buktikan nanti dengan melihat kolom `password` di phpMyAdmin.

2.3.5) Buat seeder pengaturan, kategori, dan alat

Buka terminal ketikkan perintah membuat seeder :

```
php artisan make:seeder PengaturanSeeder
```

```
php artisan make:seeder KategoriSeeder
```

```
php artisan make:seeder AlatSeeder
```

Buka file `database\seeders\PengaturanSeeder.php` :

```

9  class PengaturanSeeder extends Seeder
10 {
11     public function run(): void
12     {
13         $daftarPengaturan = [
14             ['kunci' => 'tarif_denda_harian', 'nilai' => '5000'],
15             ['kunci' => 'default_hari_pinjam', 'nilai' => '7'],
16             ['kunci' => 'maks_hari_pinjam', 'nilai' => '30'],
17             ['kunci' => 'nama_sekolah', 'nilai' => 'SMK Negeri 1 Contoh'],
18         ];
19
20         foreach ($daftarPengaturan as $data) {
21             Pengaturan::firstOrCreate(
22                 ['kunci' => $data['kunci']],
23                 ['nilai' => $data['nilai']]
24             );
25         }
26     }
27 }

```

Ganti nilai `nama_sekolah` dengan nama sekolah Anda sendiri, karena nilai itu akan tercetak sebagai kop pada seluruh laporan PDF.

Buka file `database\seeders\KategoriSeeder.php` :

```
9 class KategoriSeeder extends Seeder
10 {
11     public function run(): void
12     {
13         $daftarKategori = [
14             ['nama' => 'Perkakas Tangan', 'deskripsi' => 'Obeng, tang, kunci, palu'],
15             ['nama' => 'Alat Ukur', 'deskripsi' => 'Multimeter, jangka sorong, mistar baja'],
16             ['nama' => 'Perangkat Jaringan', 'deskripsi' => 'Switch, router, tang crimping'],
17             ['nama' => 'Perangkat Audio Visual', 'deskripsi' => 'Proyektor, kamera, tripod'],
18         ];
19
20         foreach ($daftarKategori as $data) {
21             Kategori::firstOrCreate(['nama' => $data['nama']], $data);
22         }
23     }
24 }
```

Buka file `database\seeders\AlatSeeder.php` :

```
10 class AlatSeeder extends Seeder
11 {
12     public function run(): void
13     {
14         $daftarAlat = [
15             ['kode_alat' => 'PKT-001', 'nama' => 'Obeng Plus Set', 'kategori' => 'Perkakas Tangan', 'stok' => 10],
16             ['kode_alat' => 'PKT-002', 'nama' => 'Tang Kombinasi', 'kategori' => 'Perkakas Tangan', 'stok' => 8],
17             ['kode_alat' => 'PKT-003', 'nama' => 'Kunci Pas Set', 'kategori' => 'Perkakas Tangan', 'stok' => 6],
18             ['kode_alat' => 'AUK-001', 'nama' => 'Multimeter Digital', 'kategori' => 'Alat Ukur', 'stok' => 12],
19             ['kode_alat' => 'AUK-002', 'nama' => 'Jangka Sorong', 'kategori' => 'Alat Ukur', 'stok' => 9],
20             ['kode_alat' => 'AUK-003', 'nama' => 'Mistar Baja 30 cm', 'kategori' => 'Alat Ukur', 'stok' => 15],
21             ['kode_alat' => 'JAR-001', 'nama' => 'Tang Crimping RJ45', 'kategori' => 'Perangkat Jaringan', 'stok' => 10],
22             ['kode_alat' => 'JAR-002', 'nama' => 'LAN Tester', 'kategori' => 'Perangkat Jaringan', 'stok' => 5],
23             ['kode_alat' => 'JAR-003', 'nama' => 'Switch 8 Port', 'kategori' => 'Perangkat Jaringan', 'stok' => 4],
24             ['kode_alat' => 'AVI-001', 'nama' => 'Proyektor Portabel', 'kategori' => 'Perangkat Audio Visual', 'stok' => 3],
25             ['kode_alat' => 'AVI-002', 'nama' => 'Tripod Kamera', 'kategori' => 'Perangkat Audio Visual', 'stok' => 6],
26             ['kode_alat' => 'AVI-003', 'nama' => 'Kamera Mirrorless', 'kategori' => 'Perangkat Audio Visual', 'stok' => 2],
27         ];
28
29         foreach ($daftarAlat as $data) {
30             $kategori = Kategori::where('nama', $data['kategori'])->first();
31
32             Alat::firstOrCreate(
33                 ['kode_alat' => $data['kode_alat']],
34                 [
35                     'kategori_id' => $kategori->id,
36                     'nama' => $data['nama'],
37                     'stok' => $data['stok'],
38                     'stok_tersedia' => $data['stok'],
39                     'kondisi' => 'baik',
40                 ]
41             );
42         }
43     }
44 }
```

Saat pertama kali dibuat, `stok_tersedia` selalu sama dengan `stok`, karena belum ada satu unit pun yang dipinjam.

Jangan lupa menambahkan pernyataan `use` di bagian atas tiap berkas seeder, misalnya `use App\Models\Kategori;` dan `use App\Models\Alat;`.

2.3.6) Atur urutan pemanggilan seeder

Buka file `database/seeder/DatabaseSeeder.php`:

```
5 use Illuminate\Database\Seeder;
6
7 class DatabaseSeeder extends Seeder
8 {
9     public function run(): void
10    {
11        $this->call([
12            PeranIzinSeeder::class,
13            PenggunaSeeder::class,
14            PengaturanSeeder::class,
15            KategoriSeeder::class,
16            AlatSeeder::class,
17        ]);
18    }
19 }
```

Urutannya tidak boleh diacak. `PenggunaSeeder` memanggil `syncRoles()`, jadi perannya harus sudah ada. `AlatSeeder` mencari kategori berdasarkan nama, jadi kategorinya harus sudah ada.

Seeder untuk transaksi contoh belum dibuat sekarang, karena tabel peminjaman baru akan diisi lewat antarmuka.

2.3.7) Jalankan dan periksa

Pada terminal jalan migrate serta seednya dan reset cache permission :

```
php artisan migrate:fresh --seed
php artisan permission:cache-reset
```

Perintah kedua wajib dijalankan setiap kali daftar izin berubah. Paket otorisasi menyimpan daftar izin di cache selama 24 jam, dan tanpa penyegaran, izin baru Anda tidak akan terbaca sampai besok.

Periksa di phpMyAdmin:

Tabel	Jumlah baris yang diharapkan
permissions	13
roles	3
role_has_permissions	13
model_has_roles	3
users	3
pengaturan	4
kategori	4
alat	12

2.4 Autentikasi Fortify, Layout, dan Dasbor Tiga Peran

2.4.1) Matikan fitur Fortify yang tidak dipakai

Buka `config/fortify.php` dan sesuaikan empat bagian berikut:

```
'guard' => 'web',  
'username' => 'username',  
'home' => '/dasbor',  
'features' => [  
    // Seluruh fitur bawaan dimatikan.  
    // Login dan logout tetap tersedia karena bukan bagian dari daftar ini.  
],
```

Isian `'features' => []` mematikan registrasi mandiri, reset password, verifikasi email, autentikasi dua faktor, dan pembaruan profil. Semuanya di luar ruang lingkup, dan mematakannya sekarang mencegah munculnya halaman yang tidak pernah Anda buat tampilannya.

2.4.2) Tulis logika autentikasi

Buka `app/Providers/FortifyServiceProvider.php`. Ganti seluruh isinya:

```
5 use App\Http\Responses\LoginResponse;  
6 use App\Models\LogAktivitas;  
7 use App\Models\User;  
8 use Illuminate\Http\Request;  
9 use Illuminate\Support\Facades\Hash;  
10 use Illuminate\Cache\RateLimiting\Limit;  
11 use Illuminate\Support\Facades\RateLimiter;  
12 use Illuminate\Support\ServiceProvider;  
13 use Illuminate\Validation\ValidationException;  
14 use Laravel\Fortify\Contracts\LoginResponse as KontrakLoginResponse;  
15 use Laravel\Fortify\Fortify;  
16  
17 class FortifyServiceProvider extends ServiceProvider  
18 {  
19     public function register(): void  
20     {  
21         $this->app->singleton(KontrakLoginResponse::class, LoginResponse::class);  
22     }  
23  
24     public function boot(): void  
25     {  
26         Fortify::username('username');  
27  
28         Fortify::loginView(fn () => view('auth.login'));  
29  
30         Fortify::authenticateUsing(function (Request $request) {  
31             return $this->periksaKredensial($request);  
32         });  
33  
34         RateLimiter::for('login', function (Request $request) {  
35             // Karena Anda menggunakan 'username' untuk login:  
36             $identifier = (string) $request->username;  
37  
38             // Batasi 5 kali percobaan gagal per menit berdasarkan username dan IP Address  
39             return Limit::perMinute(5)->by($identifier . $request->ip());  
40         });  
41     }  
}
```

```

33 private function periksaKredensial(Request $request): ?User
34 {
35     $pengguna = User::where('username', $request->username)->first();
36
37     // Cabang 1: user tidak ditemukan atau password tidak cocok.
38     if (! $pengguna || ! Hash::check($request->password, $pengguna->password)) {
39         $this->catatLoginGagal($request);
40
41         return null;
42     }
43
44     // Cabang 2: user ditemukan tetapi akunnya dinonaktifkan.
45     if (! $pengguna->is_aktif) {
46         $this->catatLoginGagal($request, $pengguna->id);
47
48         throw ValidationException::withMessages([
49             'username' => 'Akun Anda dinonaktifkan. Hubungi administrator.',
50         ]);
51     }
52
53     // Cabang 3: kredensial cocok dan akun aktif.
54     LogAktivitas::create([
55         'user_id' => $pengguna->id,
56         'aksi' => 'login',
57         'tabel_tujuan' => 'users',
58         'deskripsi' => 'Pengguna ' . $pengguna->username . ' berhasil masuk',
59         'ip_address' => $request->ip(),
60     ]);
61
62     return $pengguna;
63 }

```

```

65 private function catatLoginGagal(Request $request, ?int $penggunaId = null): void
66 {
67     LogAktivitas::create([
68         'user_id' => $penggunaId,
69         'aksi' => 'login_gagal',
70         'tabel_tujuan' => 'users',
71         'deskripsi' => 'Percobaan masuk gagal untuk username ' . $request->username,
72         'ip_address' => $request->ip(),
73     ]);
74 }
75 }

```

Bagian yang paling sering salah: cabang akun nonaktif harus melempar `ValidationException`, bukan sekadar mengembalikan `null`. Bila hanya mengembalikan `null`, Fortify menampilkan pesan gagal generik.

Perhatikan juga bahwa `catatLoginGagal()` mengisi `user_id` dengan `null` pada cabang pertama. Inilah alasan kolom itu dibuat boleh kosong: saat username-nya tidak dikenal, memang tidak ada user yang bisa dicatat.

2.4.3) Buat penentu tujuan setelah login

Buat berkas baru `app/Http/Responses/LoginResponse.php`:

```
3 namespace App\Http\Responses;
4
5 use Laravel\Fortify\Contracts\LoginResponse as KontrakLoginResponse;
6
7 class LoginResponse implements KontrakLoginResponse
8 {
9     public function toResponse($request)
10     {
11         $pengguna = $request->user();
12
13         if ($pengguna->hasRole('admin')) {
14             return redirect()->route('admin.dasbor');
15         }
16
17         if ($pengguna->hasRole('petugas')) {
18             return redirect()->route('petugas.dasbor');
19         }
20
21         return redirect()->route('peminjam.dasbor');
22     }
23 }
```

2.4.4) Susun route

Buka `routes/web.php`:

```
3 use Illuminate\Support\Facades\Route;
4
5 Route::get('/', function () {
6     return redirect()->route('login');
7 });
8
9 Route::middleware(['auth'])->group(function () {
10
11     Route::get('/admin/dasbor', function () {
12         return view('dasbor.admin');
13     })->middleware('role:admin')->name('admin.dasbor');
14
15     Route::get('/petugas/dasbor', function () {
16         return view('dasbor.petugas');
17     })->middleware('role:petugas')->name('petugas.dasbor');
18
19     Route::get('/peminjam/dasbor', function () {
20         return view('dasbor.peminjam');
21     })->middleware('role:peminjam')->name('peminjam.dasbor');
22
23 });
```

Route `login` dan `logout` tidak perlu ditulis. Keduanya sudah didaftarkan Fortify secara otomatis.

Ketiga dasbor untuk sementara masih berupa halaman kosong. Isinya akan bertambah seiring modul-modul berikutnya selesai.

2.4.5) Buat layout utama

Buat file `resources/views/layouts/utama.blade.php`:

```
1 <!DOCTYPE html>
2 <html lang="id">
3
4 <head>
5     <meta charset="utf-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1">
7     <title>@yield('judul', 'Peminjaman Alat')</title>
8     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
9 </head>
10
11 <body class="bg-light">
12
13     @include('layouts.navbar')
14
15     <div class="container py-4">
16         @if (session('sukses'))
17             <div class="alert alert-success">{{ session('sukses') }}</div>
18         @endif
19
20         @if (session('gagal'))
21             <div class="alert alert-danger">{{ session('gagal') }}</div>
22         @endif
23
24         @yield('konten')
25     </div>
26
27     <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
28 </body>
29
30 </html>
```

Buat file `resources/views/layouts/navbar.blade.php`:

```
1 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
2     <div class="container">
3         <a class="navbar-brand" href="{{ url('/') }}">Peminjaman Alat</a>
4
5         <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#menuUtama">
6             <span class="navbar-toggler-icon"></span>
7         </button>
8
9         <div class="collapse navbar-collapse" id="menuUtama">
10             <ul class="navbar-nav me-auto">
11                 @can('kategori.kelola')
12                     <li class="nav-item"><a class="nav-link" href="#">Kategori</a></li>
13                 @endcan
14                 @can('alat.kelola')
15                     <li class="nav-item"><a class="nav-link" href="#">Alat</a></li>
16                 @endcan
17                 @can('user.kelola')
18                     <li class="nav-item"><a class="nav-link" href="#">Pengguna</a></li>
19                 @endcan
20                 @can('peminjaman.setujui')
21                     <li class="nav-item"><a class="nav-link" href="#">Persetujuan</a></li>
22                 @endcan
23                 @can('alat.lihat')
24                     <li class="nav-item"><a class="nav-link" href="#">Katalog Alat</a></li>
25                 @endcan
26             </ul>
27         </div>
28     </div>
29 </nav>
```

```

28  ✓      @auth
29  ✓      <ul class="navbar-nav">
30  ✓          <li class="nav-item">
31  ✓              <span class="navbar-text me-3">{{ auth()→user()→nama }}</span>
32  ✓          </li>
33  ✓          <li class="nav-item">
34  ✓              <form method="POST" action="{{ route('logout') }}">
35  ✓                  @csrf
36  ✓                  <button type="submit" class="btn btn-sm btn-outline-light">Keluar</button>
37  ✓              </form>
38  ✓          </li>
39  ✓      </ul>
40  ✓      @endauth
41  ✓  </div>
42  </div>
43  </nav>

```

Menu di layout ini dibungkus `@can`, bukan `@if` yang membandingkan nama peran. Akibatnya, menu otomatis menyesuaikan izin yang Anda seed — bila izin berubah, menu ikut berubah tanpa menyentuh berkas ini.

Tautan menu masih berisi tanda pagar. Alamat sebenarnya diisi pada job sheet yang membuat modulnya.

2.4.6) Buat halaman login

Buat file `resources/views/auth/login.blade.php`:

```

1  @extends('layouts.utama')
2
3  @section('judul', 'Masuk')
4
5  @section('konten')
6      <div class="row justify-content-center">
7          <div class="col-md-5">
8              <div class="card shadow-sm">
9                  <div class="card-body p-4">
10                     <h4 class="card-title mb-4 text-center">Masuk ke Sistem</h4>
11
12                     <form method="POST" action="{{ route('login') }}">
13                         @csrf
14
15                         <x-input name="username" label="Nama Pengguna" required autofocus />
16
17                         <x-input name="password" label="Kata Sandi" type="password" required value="" />
18
19                         <button type="submit" class="btn btn-primary w-100">Masuk</button>
20                     </form>
21                 </div>
22             </div>
23         </div>
24     </div>
25 @endsection
26

```

Buat file `resources\views\components\input.blade.php` :

```
1 @props(['name', 'label', 'type' => 'text', 'value' => ""])
2
3 <div class="mb-3">
4     <label for="{{ $name }}" class="form-label">{{ $label }}</label>
5     <input type="{{ $type }}"
6         class="form-control @error($name) is-invalid @enderror"
7         id="{{ $name }}"
8         name="{{ $name }}"
9         value="{{ old($name, $value) }}"
10        {{ $attributes }}
11    >
12
13    @error($name)
14        <div class="invalid-feedback">
15            {{ $message }}
16        </div>
17    @enderror
18 </div>
```

2.4.7) Buat tiga halaman dasbor

Buat `resources\views\dasbor\admin.blade.php`:

```
1 @extends('layouts.utama')
2
3 @section('judul', 'Dasbor Admin')
4
5 @section('konten')
6     <h4>Dasbor Admin</h4>
7     <p class="text-muted">Selamat datang, {{ auth()->user()->nama }}.</p>
8 @endsection
```

Buat dua berkas serupa dengan mengganti judulnya: `dasbor/petugas.blade.php` dan `dasbor/peminjam.blade.php`.

2.4.8) Uji seluruh cabang login

Jalankan `php artisan serve`, lalu buka <http://127.0.0.1:8000>.

Percobaan	Masukan	Hasil yang harus muncul
1	admin / password123	Masuk ke Dasbor Admin, menu berisi Kategori, Alat, Pengguna
2	petugas / password123	Masuk ke Dasbor Petugas, menu hanya berisi Persetujuan
3	peminjam / password123	Masuk ke Dasbor Peminjam, menu hanya berisi Katalog Alat
4	admin / salah	Tetap di halaman login, muncul pesan kredensial tidak cocok
5	Akun nonaktif	Muncul pesan “Akun Anda dinonaktifkan”
6	Salah password 6 kali beruntun	Ditolak sementara oleh pembatasan percobaan Fortify

Halaman Login :

Peminjaman Alat

Masuk ke Sistem

Nama Pengguna

Kata Sandi

Masuk

Request Timeline Views 5 Queries 2 Gate 5 12.x 26MB 914ms 2 GET /login

Halaman dasbor admin :

Peminjaman Alat Kategori Alat Pengguna Administrator **Keluar**

Dasbor Admin

Selamat datang, Administrator.

12.x 28MB 485ms 2 GET /admin/dasbor

Halaman dasbor petugas :

Peminjaman Alat Persetujuan Petugas Laboratorium **Keluar**

Dasbor Petugas

Selamat datang, Petugas Laboratorium.

12.x 28MB 578ms 2 GET /petugas/dasbor

Halaman dasbor peminjam :

